

L'Era dei Transformers

“

"The Transformer is a general-purpose computer that is differentiable, and it is the most powerful tool we have for modeling sequences." – Andrej Karpathy

”



by Daniele Mantovani

Il Limite di Word2Vec: Embeddings Statici

Word2Vec rappresenta ogni parola come un vettore numerico fisso in uno spazio multidimensionale. Una volta addestrato, questa "lookup table" statica associa un unico embedding a ogni parola, rendendo il suo vettore immutabile indipendentemente dal contesto di utilizzo.

"I went to the **bank** to deposit some money."



"I walked along the river **bank**."



Per Word2Vec, la parola **bank** ha lo stesso identico vettore numerico in entrambi i casi. Questo modello non è in grado di cogliere le diverse sfumature di significato che una parola può assumere a seconda della frase, perdendo completamente il significato contestuale.

“

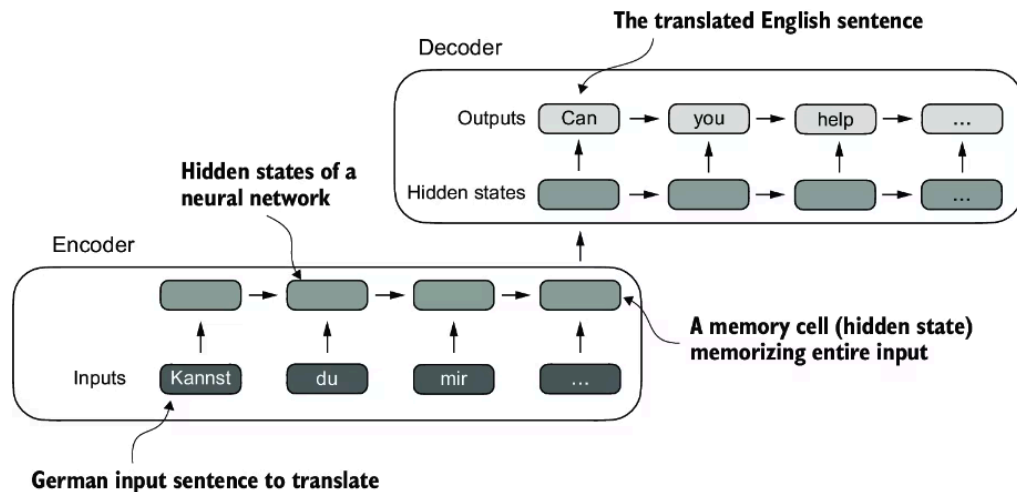
Per comprendere veramente il linguaggio, ci serve un modello che legga e interpreti la frase intera, non solo le parole isolate, per generare embedding contestuali e dinamici.

”

Reti Neurali Ricorrenti (RNN)

Le Reti Neurali Ricorrenti (RNN) sono architetture neurali progettate specificamente per elaborare dati sequenziali, come serie temporali, testo o audio. A differenza dei modelli feed-forward, le RNN introducono un concetto di "**memoria**" interna.

Le RNN prendono in ingresso gli embedding statici (es. Word2Vec), un elemento alla volta, in una sequenza. La loro caratteristica distintiva è la capacità di mantenere uno stato interno (**hidden state h**) che si aggiorna ad ogni passo temporale, incorporando informazioni dagli input precedenti.



*Immagina l'encoder come un traduttore umano che legge una frase. Le singole parole scritte sulla carta sono i token. Prima di poter tradurre, il traduttore trasforma ogni parola scritta in un **concetto mentale** (l'embedding). Solo dopo aver elaborato questi concetti in ordine, il traduttore forma un'idea completa della frase (il vettore di contesto) da passare a chi dovrà scriverla nell'altra lingua.*

$$h(t) = f(x(t), h(t - 1))$$

Context Embedding è l'ultimo hidden state generato (nelle RNN senza meccanismo di attenzione)

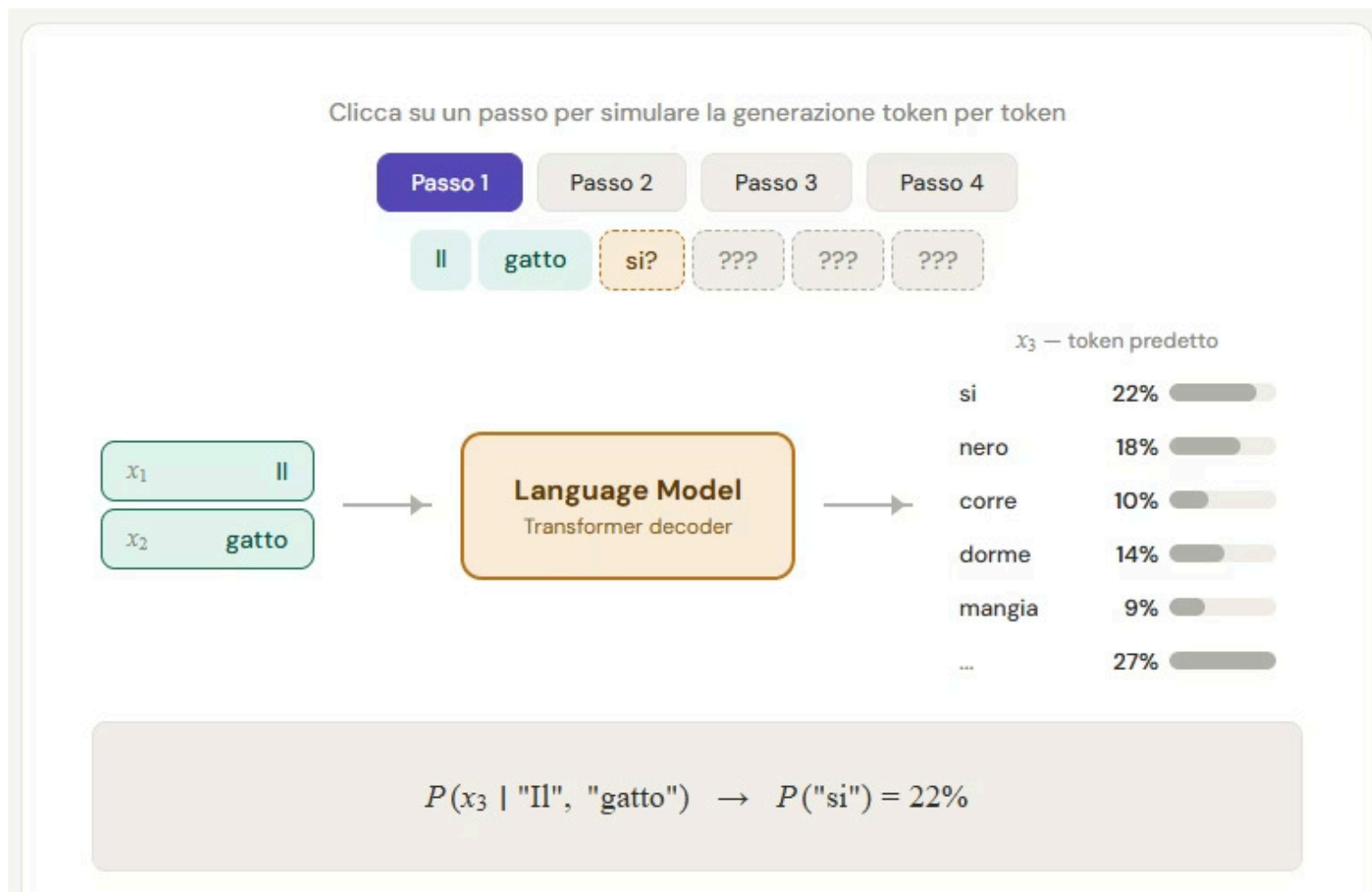
Questo meccanismo permette alle RNN di comprendere e generare sequenze, catturando le dipendenze temporali nei dati e superando il limite degli embedding statici di Word2Vec che non consideravano il contesto.

“

Poiché l'intera frase deve essere "compressa" in un unico vettore finale, le RNN soffrono del problema delle **dipendenze a lungo raggio** (vanishing gradient): lo stato nascosto finale tende a "dimenticare" le informazioni dei primi token se la frase è molto lunga

”

Cos'è un Language Model



[Cos'è un Language Model](#)

Self-Attention

Idea chiave: ogni token può **interrogare direttamente tutti gli altri** in un singolo passo, senza passare l'informazione in sequenza.

Esempio: "Il gatto si sedette sul tappeto"

- "sedette" → 40% attenzione a "gatto" (il soggetto)
- "sul" → 55% attenzione a "tappeto" (il nome che regge)
- "gatto" → guarda sia "sedette" sia "tappeto"

Come funziona: per ogni token i , il modello calcola un peso α_{ij} verso ogni altro token j

- Questo peso esprime quanto il token j è rilevante per il token i
- I pesi sono ottenuti tramite dot product tra i vettori Query (token i) e Key (token j), normalizzati con Softmax
- I pesi cambiano in base al contesto → rappresentazioni **contestualizzate**

Vantaggi rispetto alle RNN:

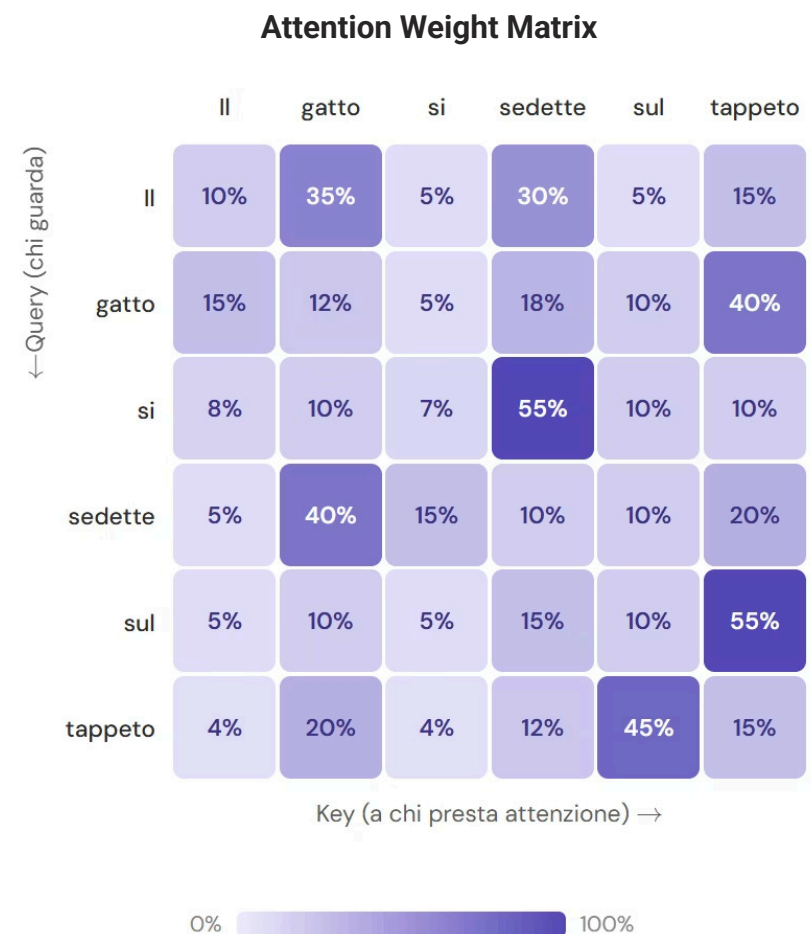
- Path length** $O(1)$ — accesso diretto a qualsiasi token, contro $O(n)$ delle RNN
- Parallelismo completo** — un'unica moltiplicazione matriciale, GPU sfruttate appieno
- Interpretabilità** — i pesi α_{ij} mostrano cosa il modello "guarda"

Il prezzo: complessità $O(n^2)$

- Ogni token interagisce con ogni altro → costo quadratico
- $n = 512 \rightarrow \sim 262k$ coppie | $n = 4096 \rightarrow \sim 16.8M$ coppie

Perchè dividere per $\sqrt{d_k}$

- Il dot product $q \cdot k$ cresce in magnitudine con la dimensionalità d_k
- Valori grandi → softmax satura → gradienti piccoli (vanishing)
- Dividere per $\sqrt{d_k}$ normalizza la varianza del dot product a ~ 1



Come leggere la matrice

Ogni **riga** corrisponde a un token **query** — il token che "guarda" gli altri. Ogni

colonna corrisponde a un token **key** — il token che viene guardato.

La cella (riga i , colonna j) contiene il peso α_{ij} : quanta attenzione il token i presta al token j . Ogni riga somma a 100% (distribuzione softmax).

Colore più intenso = attenzione maggiore.

[Query, Key, Value — Meccanismo della Self-Attention](#)

[Attention Weight Matrix](#)

Positional Encoding

La self-attention senza informazione posizionale è **permutation-equivariant**: se permutiamo i token in input, le uscite si permutano allo stesso modo — il modello non ha una nozione intrinseca d'ordine. 'il gatto mangia il pesce' e 'il pesce mangia il gatto' generano rappresentazioni diverse (perché i vettori Value sono diversi), ma la *struttura dei pesi di attenzione* non dipende dalle posizioni. Serve iniettare l'ordine esplicitamente.

Il problema:

- Il dot product $q_i^T k_j$ dipende solo dal contenuto, non dalla posizione dei token
- Se permutiamo i token in input, i pesi di attenzione non cambiano → il modello è cieco all'ordine
- L'ordine delle parole è fondamentale per il significato — serve iniettarlo esplicitamente

La soluzione:

- Sommare all'embedding di contenuto un vettore che codifica la posizione: $\text{input}_i = \text{embedding}(x_i) + \text{PE}(\text{pos}_i)$
- Il vettore posizionale ha la stessa dimensione d_{model} dell'embedding — zero parametri aggiuntivi nella versione sinusoidale
- Due approcci principali: sinusoidale (Vaswani 2017) e appreso (BERT, GPT-2)

[Infografica](#)

RoPE – Rotary Position Embedding

I modelli moderni (LLaMA, Mistral, DeepSeek, Qwen) usano RoPE: invece di sommare la posizione, **ruotano** i vettori Q e K.

L'idea chiave (Su et al., 2021):

- Dividi Q e K in coppie di dimensioni consecutive: $(q_0, q_1), (q_2, q_3), \dots$
- Ogni coppia è un punto in un piano 2D — ruotala di un angolo $\theta_i \cdot pos$
- Angolo piccolo per coppie ad alta dimensione (bassa frequenza), grande per coppie a bassa dimensione

Perché è meglio del PE additivo:

- Il dot product $q_m^T k_n$ dipende solo da $m - n$ (distanza relativa), non dalle posizioni assolute
- Nessun parametro aggiuntivo — la rotazione è deterministica
- Applicato **dentro** ogni attention layer, non solo all'inizio — l'info posizionale non si "diluiscie" attraverso i layer
- Estendibile a context length più lunghe con tecniche come NTK-aware scaling e YaRN

[Infografica](#)

Apprendimento delle Matrici

Le matrici W_Q , W_K , W_V non sono progettate a mano — sono parametri appresi dal training che danno alla self-attention tre capacità distinte.

Il ruolo di ciascuna matrice:

- W_Q codifica "cosa sto cercando" — il verbo cerca il soggetto, la preposizione cerca il nome che regge
- W_K codifica "cosa offro" — categoria grammaticale, ruolo tematico, tipo di relazione
- W_V codifica "quale informazione trasporto" — il contenuto semantico utile da propagare

Perché ne servono esattamente tre:

- $W_Q \neq W_K \rightarrow$ **asimmetria**: "sedette" guarda molto "gatto", ma "gatto" può guardare altrove
- W_V separata $\rightarrow$ **disaccoppiamento**: il criterio con cui trovo un token rilevante è diverso dall'informazione che ne estraggo



La self-attention non è un algoritmo "programmato" per trovare soggetti e verbi — è una struttura **flessibile** che il training riempie di significato.

Ad esempio se lo stesso Transformer venisse addestrato su musica anziché testo, le matrici W_* imparerebbero pattern armonici, relazioni tra note, strutture ritmiche — il meccanismo è lo stesso, il contenuto cambia.



[Perché servono tre matrici W — Self-Attention](#)

Causal Self-Attention

Nei modelli autogressivi (GPT, LLaMA, Claude), ogni token può guardare solo i token che lo precedono — mai quelli futuri. Senza questo vincolo, il modello "barerebbe" durante il training leggendo la risposta prima di generarla.

Come funziona:

- La matrice di attenzione viene mascherata con una maschera triangolare inferiore
- Le posizioni future ricevono $-\infty$ prima di softmax $\rightarrow$ dopo softmax diventano 0
- Stessa meccanica dell'attention mask per il padding, ma applicata ai token futuri anziché ai token vuoti

Cosa cambia rispetto alla self-attention completa:

- "Il" può guardare solo sé stesso $\rightarrow$ 100% su "Il"
- "gatto" può guardare "Il" e sé stesso $\rightarrow$ distribuisce tra i due
- "sedette" può guardare "Il", "gatto", "si" e sé stesso $\rightarrow$ il 57% va a "gatto" (il soggetto)
- L'ultima riga ("tappeto") è identica alla self-attention completa — vede tutto il contesto precedente

	Il	gatto	si	sedette	sul	tappeto
Il	100%	—	—	—	—	—
gatto	56%	44%	—	—	—	—
si	32%	40%	28%	—	—	—
sedette	7%	57%	21%	14%	—	—
sul	11%	22%	11%	33%	22%	—
tappeto	4%	20%	4%	12%	45%	15%

[Causal Self-Attention](#)

Architettura Transformer Completa

Il Transformer originale (Vaswani et al., 2017) è composto da due blocchi simmetrici ma con ruoli distinti.

Encoder — "capire l'input":

- Riceve la sequenza di input e ne costruisce una rappresentazione contestuale ricca
- La self-attention è bidirezionale: ogni token può "guardare" tutti gli altri token della sequenza
- L'output è una sequenza di vettori contestualizzati, uno per ogni token di input

Decoder — "generare l'output":

- Produce la sequenza di output un token alla volta, in modo autoregressivo
- La Masked Self-Attention impedisce di "guardare avanti": ogni token vede solo i precedenti
- La Cross-Attention collega decoder e encoder — le query vengono dal decoder, chiavi e valori dall'encoder

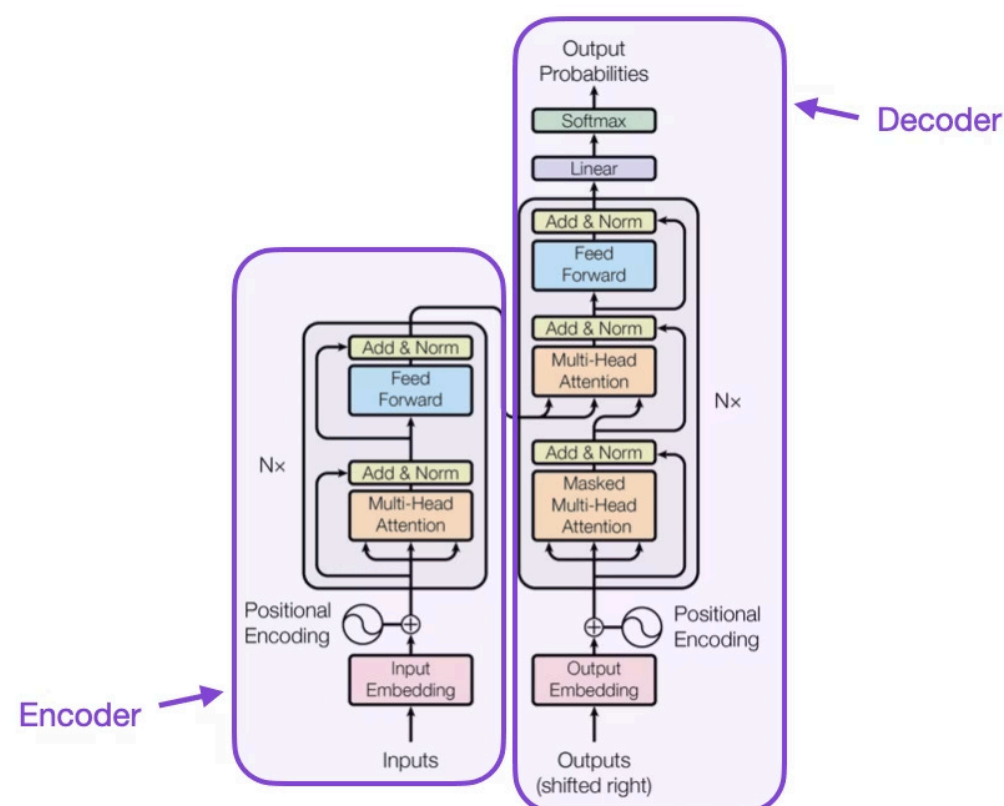
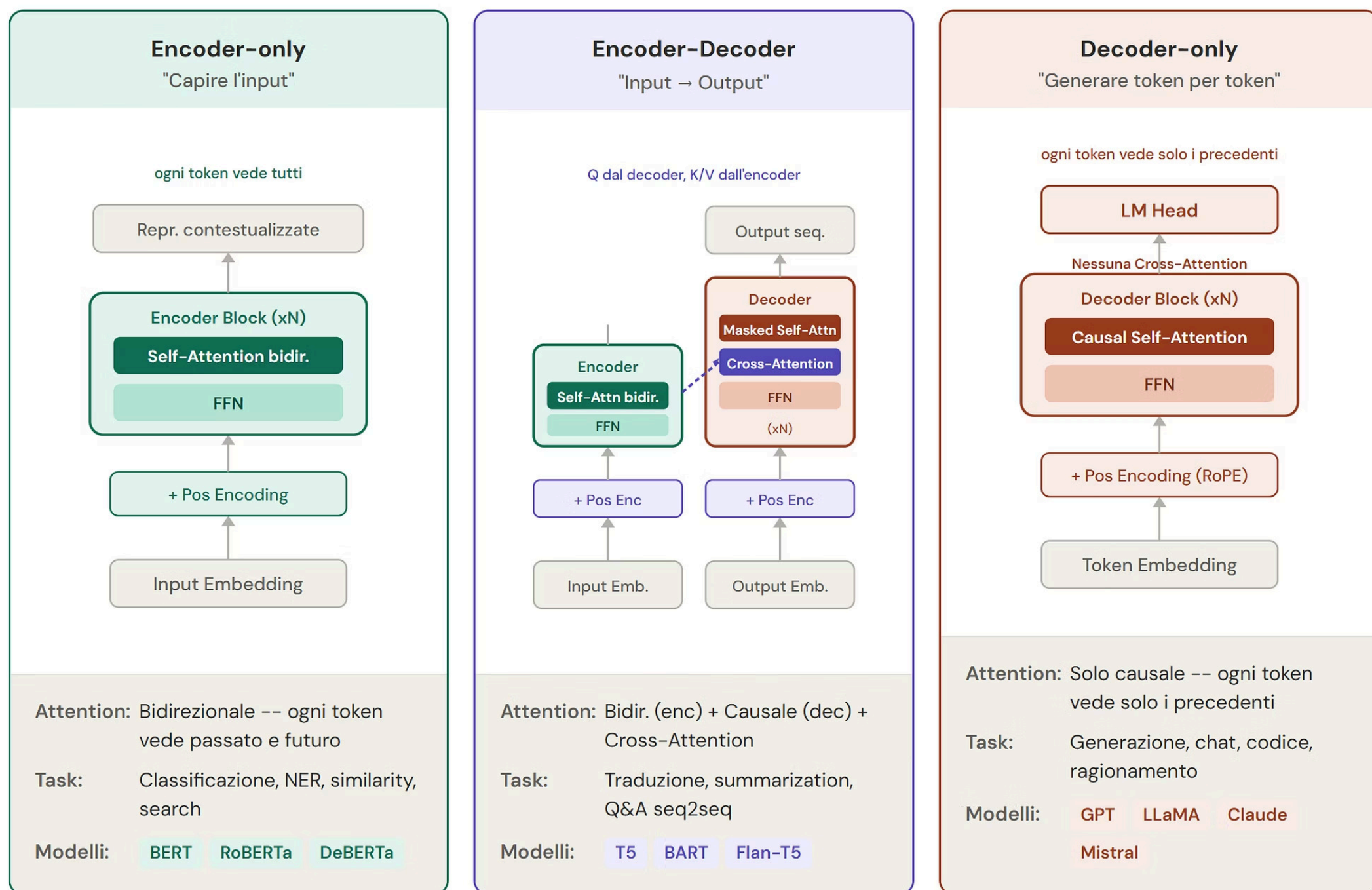


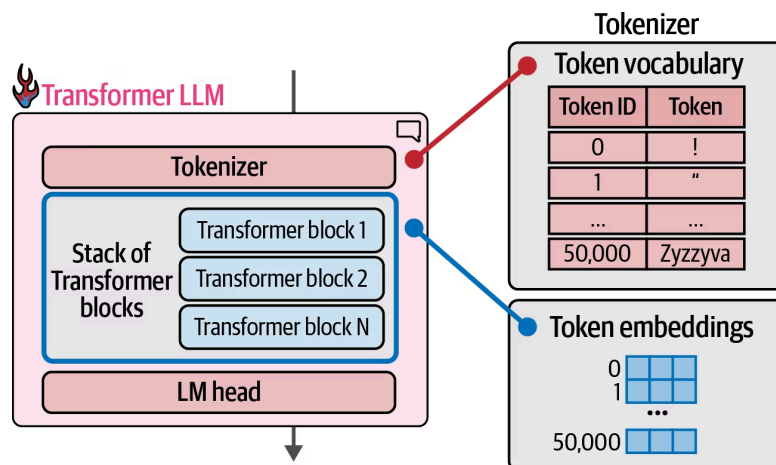
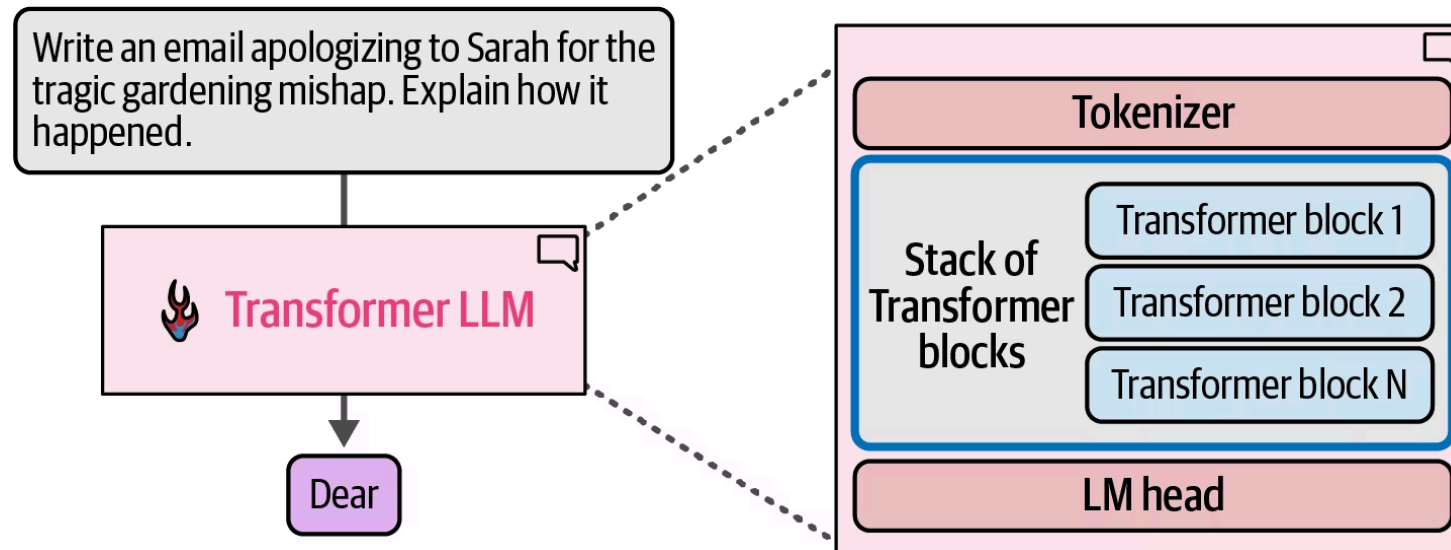
Figure 1: The Transformer - model architecture.

■ Self-Attention bidirezionale ■ Causal (Masked) Self-Attention ■ Cross-Attention

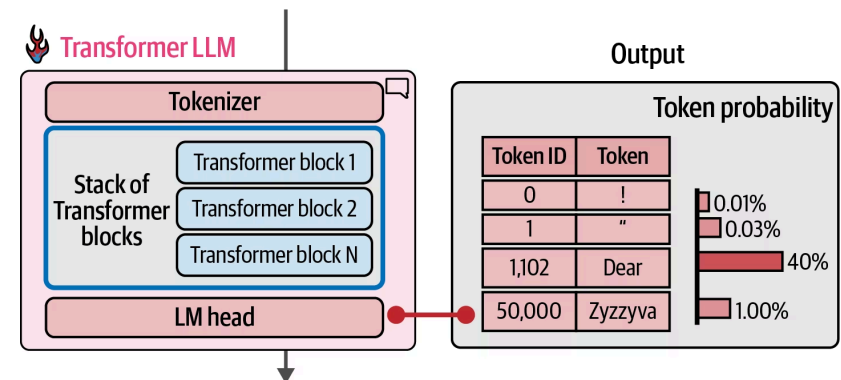


Tutti i principali LLM conversazionali (GPT, LLaMA, Claude, Mistral, DeepSeek, Qwen) sono **decoder-only**: usano solo la causal self-attention, senza encoder e senza cross-attention. Il prompt e la risposta vivono nella stessa sequenza di token.

Le componenti principali del Transformer



Il tokenizer ha un vocabolario di 50.000 **token**. Il modello ha degli **embedding** di token associati a tali elementi.



La **LM Head** è il livello di rete neurale finale che trasforma l'output del modello in punteggi di probabilità per ogni parola del vocabolario, determinando così il token successivo. E' di fatto un classificatore.

All'interno di un blocco Transformer

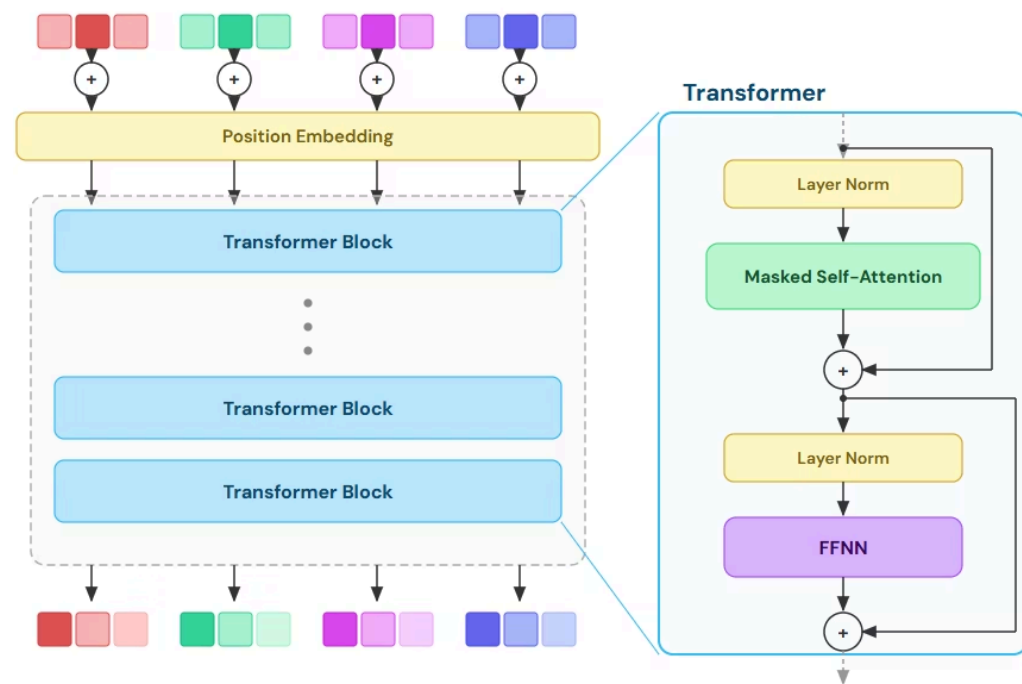
- Gli LLM Transformer sono composti da una serie di blocchi (anche oltre cento) dove avviene la maggior parte del calcolo.
- Ogni blocco processa i suoi input e passa i risultati direttamente al blocco seguente della sequenza.

Un blocco Transformer è costituito da quattro componenti che si ripetono in sequenza:

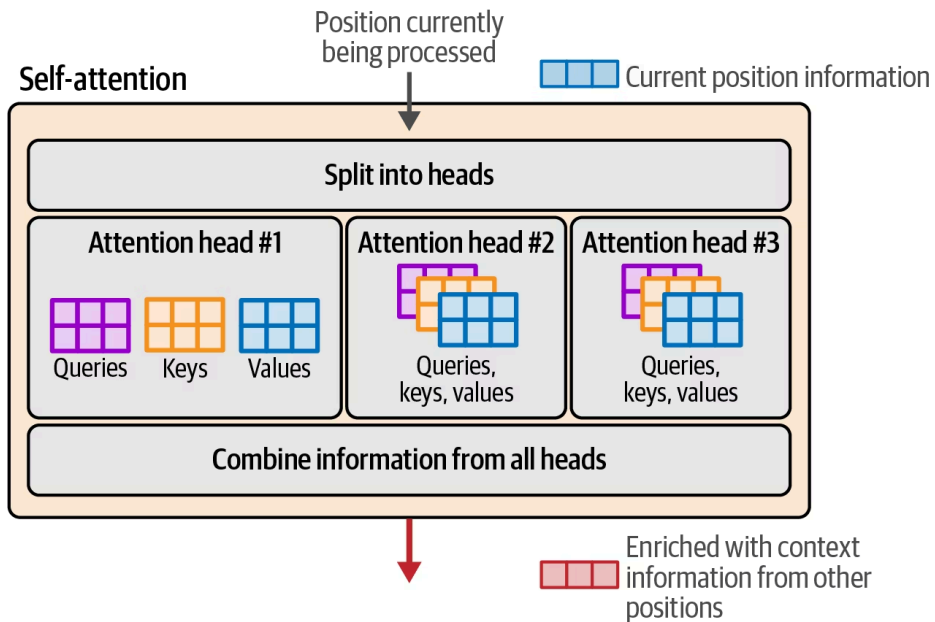
- Layer Norm
- Multi-Head Self-Attention + residual connection
- Layer Norm
- Feed-Forward Network + residual connection.

Le residual connections permettono al gradiente di fluire senza degradarsi attraverso decine di blocchi.

La Layer Norm stabilizza le attivazioni.



Multi-Head Attention



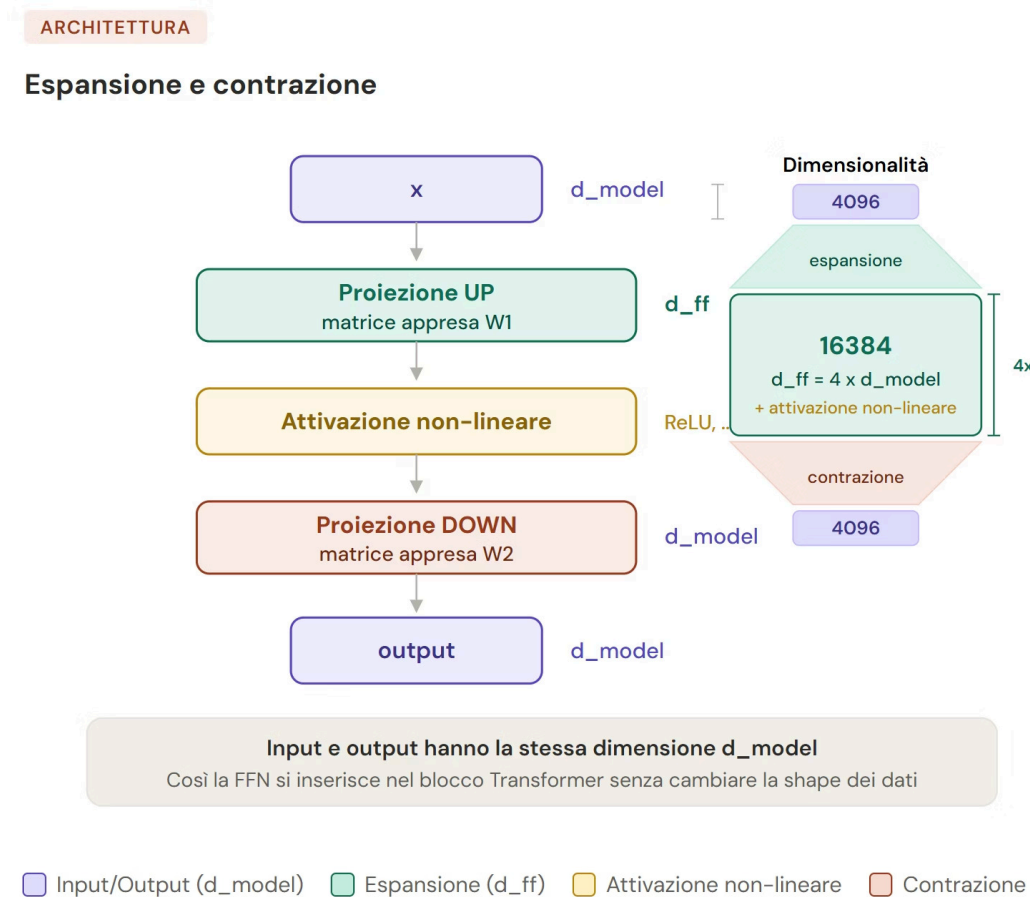
- **Elaborazione Parallela:** Il concetto fondamentale della multi-head attention consiste nel duplicare il meccanismo di attenzione ed eseguirlo più volte in parallelo. Ciascuna di queste esecuzioni parallele costituisce una "testa di attenzione" (attention head).
- **Capacità di Modellazione Aumentata:** L'utilizzo di più teste incrementa la capacità del modello di comprendere pattern complessi. Permette al modello di prestare attenzione a diversi tipi di informazioni o relazioni all'interno della sequenza di input simultaneamente.
- **Matrici Distinte:** Nella versione originale, ogni testa di attenzione possiede le proprie matrici di proiezione distinte per creare le *query*, le *key* e i *value*. Ciò significa che ogni testa calcola l'attenzione in modo unico per un dato input.
- **Flusso del Processo:** Il funzionamento può essere visualizzato come una divisione delle informazioni in ingresso, seguita da un'elaborazione parallela attraverso le diverse teste, e infine una combinazione (aggregazione) dei risultati di tutte le teste per formare l'output finale del layer di attenzione,.
- **Ottimizzazione ed Efficienza:** Per migliorare l'efficienza, specialmente nei modelli più grandi, sono state sviluppate varianti come la *Multi-query attention* (dove le matrici *key* e *value* sono condivise tra tutte le teste) e la *Grouped-query attention* (dove *key* e *value* sono condivisi tra gruppi di teste). Queste tecniche riducono la dimensione delle matrici coinvolte e migliorano la scalabilità dell'inferenza,,.

Feed-Forward Network (FFN)

Presente in ogni Transformer Block, la FFN opera su ciascuna posizione individualmente. L'attenzione mescola informazioni tra posizioni; la FFN trasforma ogni posizione, aggiungendo la non-linearità indispensabile al modello.

La FFN è un **bottleneck invertito**: prima espande il vettore a una dimensione molto più grande (d_{ff} , tipicamente $4 \times d_{model}$), applica una funzione non-lineare, e poi ricomprime alla dimensione originale.

Perché espandere? Nello spazio ad alta dimensionalità le feature sono più facili da separare e manipolare. È lo stesso principio del kernel trick nelle SVM: proiettare in uno spazio più grande rende separabili pattern che nello spazio originale sarebbero intrecciati. La non-linearità (ReLU, ...) decide quali feature "accendere" e quali silenziare per ogni input specifico. La contrazione finale riporta il risultato alla dimensione del modello, mantenendo solo l'informazione utile.



L'attention, pur contenendo la non-linearità della softmax, produce un output vincolato: una combinazione convessa dei vettori Value, cioè un punto all'interno del loro convex hull. Questo limita ciò che l'attention può esprimere da sola. La FFN rompe questo vincolo: grazie all'espansione dimensionale e all'attivazione non-lineare (ReLU/SiLU/GELU), trasforma ogni rappresentazione in modo arbitrario per posizione. L'alternanza attention (mescola informazione tra posizioni) + FFN (trasforma per posizione) è ciò che dà al Transformer la sua potenza espressiva.

- ❑ Senza FFN, il modello avrebbe molta meno capacità di trasformazione: l'attenzione mescola informazione tra posizioni, ma è la FFN che la elabora in profondità. Insieme, le due componenti formano un ciclo mix-then-transform ripetuto per decine di blocchi.

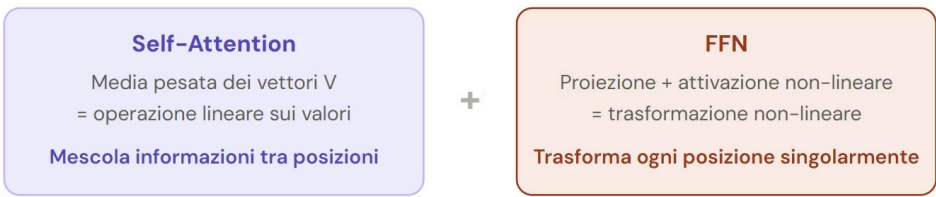
La FFN come memoria del modello

Ricerche recenti (Geva et al., 2021) hanno mostrato che la FFN funziona anche come una **memoria associativa appresa**. Ogni neurone del layer espanso si attiva in risposta a pattern specifici nell'input, e la proiezione di ritorno "inietta" l'informazione associata nella rappresentazione.

Questo significa che buona parte della **conoscenza fattuale** di un LLM — fatti, associazioni, relazioni — è codificata nei pesi delle FFN. Tecniche di *knowledge editing* come ROME (Meng et al., 2022) sfruttano questa proprietà per modificare fatti specifici nel modello agendo chirurgicamente sui pesi delle FFN, senza riaddestrare l'intero modello.

PERCHÉ SERVE

La non-linearità che l'attenzione non ha

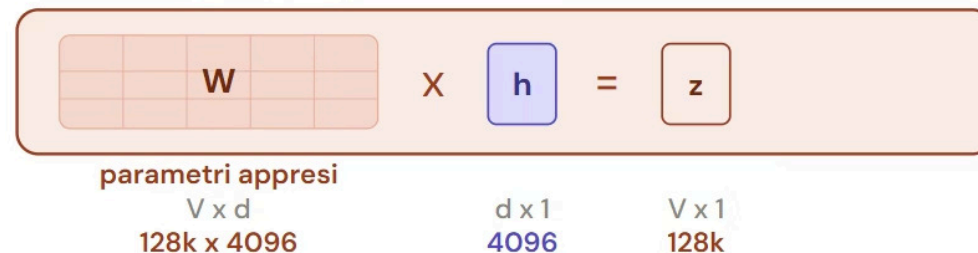


LM Head

1 Input: output dell'ultimo Transformer Block



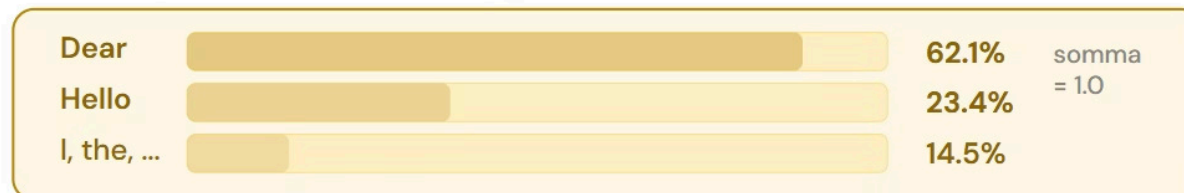
2 Proiezione lineare (matrice appresa in training)



3 Logits: uno score per ogni token del vocabolario



4 Softmax: logits --> distribuzione di probabilità



5 Sampling (o argmax)

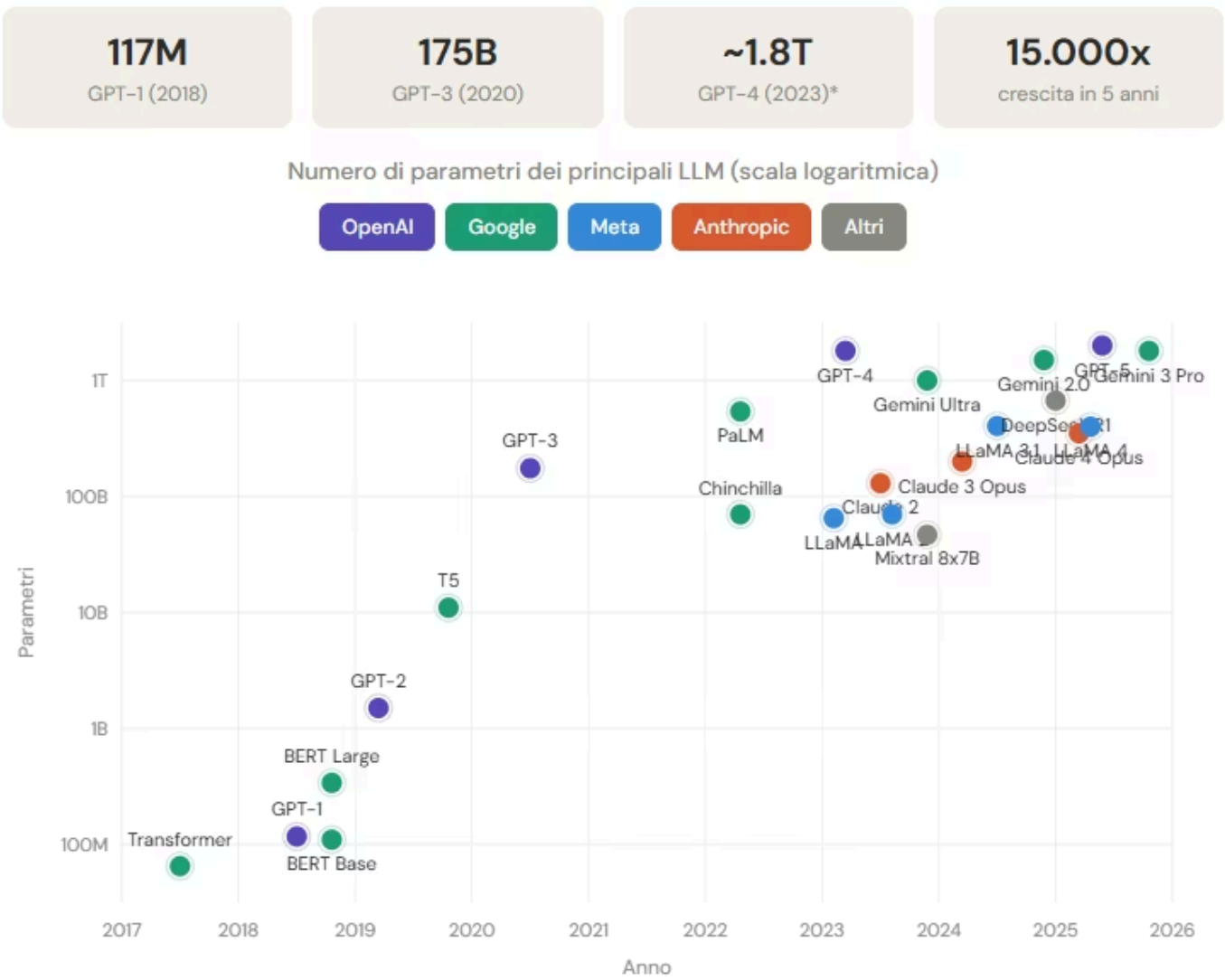


 Vettore h (hidden state) LM Head (proiezione appresa) Logits Probabilità (softmax)

📄 **Weight tying.** In molte architetture decoder-only (GPT, Llama, Gemma) la matrice W della LM Head non è un parametro indipendente: coincide con la **trasposta della matrice di embedding** di input ($W_{lm} = W_{emb}^T$). Anche in questo caso W è appresa: è la stessa matrice di embedding, addestrata durante il training e condivisa tra i due usi. Calcolare i logits equivale allora a misurare la *similarità* (prodotto scalare) tra h e ciascun embedding del vocabolario.

📄 **Quanti parametri?** La matrice della LM Head ha $V \times d_{model}$ parametri. Con vocabolario 128k e dimensione 4096, sono circa **524 milioni** di parametri, solo per questo layer. Il weight tying evita di raddoppiare questo costo condividendo i pesi con l'embedding layer.

Cosa sono i Large Language Model (LLM)



LLM Landscape

Cosa rende un Language Model "Large"

Non esiste una soglia ufficiale, ma in pratica si parla di LLM quando il modello ha **miliardi di parametri** ed è stato addestrato su **triloni di token** da web, libri, codice e conversazioni. L'architettura è la stessa self-attention che abbiamo studiato — la differenza è quantitativa, non qualitativa.

Tendenze recenti (2024-2025):

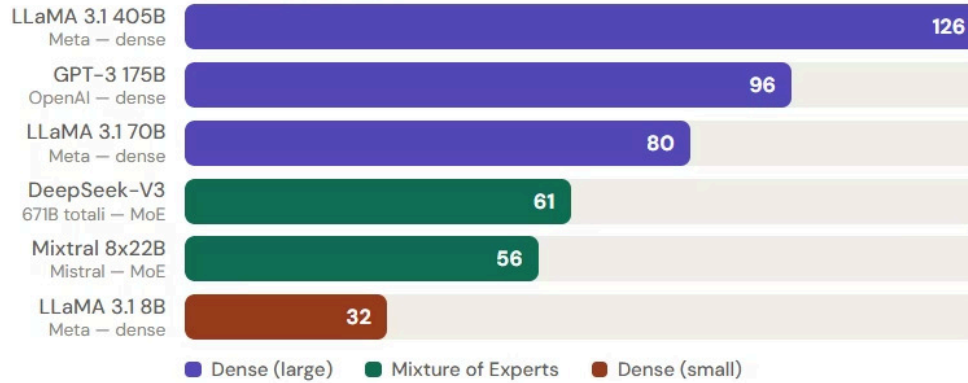
- I modelli più grandi superano i 1.000 miliardi di parametri (GPT-4, Gemini Ultra)
- Mixture of Experts (MoE): modelli con molti parametri totali ma che ne attivano solo una frazione per ogni token (es. Mixtral, DeepSeek V3)
- Modelli open-weight competitivi: LLaMA, DeepSeek, Qwen sfidano i modelli proprietari
- Reasoning models: catene di pensiero esplicite prima della risposta (o1, R1, Claude con extended thinking)
- La dimensione non è tutto — Chinchilla (70B) ha dimostrato che più dati con meno parametri può battere modelli più grandi

📌 **Il paradosso degli LLM:** un task semplice (predire la prossima parola) + scala enorme = capacità sorprendenti. Few-shot learning, ragionamento multi-step, scrittura di codice, traduzione — tutto emerge da $P(x_t | x_1, \dots, x_{t-1})$ applicato con miliardi di parametri su triloni di dati.

** I conteggi dei parametri per i modelli proprietari (GPT-4, Gemini) sono stime non ufficiali.*

Quanti Transformer Block?

NUMERO DI LAYER (TRANSFORMER BLOCK)



COME SCALA IL NUMERO DI PARAMETRI

Stima approssimata dei parametri di un Transformer denso

$$P \approx 12 \cdot L \cdot d_{\text{model}}^2$$

L = numero di layer | d_{model} = dimensione nascosta | 12 tiene conto delle 4 matrici per layer (Q, K, V, O) e dei 2 layer FFN

Piu profondità (L grande)

- Rappresentazioni via via piu astratte e composte
- Piu difficile da parallelizzare (dipendenza sequenziale tra layer)
- Rischio di vanishing/exploding gradients senza pre-norm
- Richiede tecniche come pipeline parallelism

Piu larghezza (d grande)

- Ogni layer ha piu capacità espressiva
- Piu facile da parallelizzare con tensor parallelism
- Maggior costo di memoria per le attivazioni
- Strategia preferita oltre una certa scala

Caso MoE: piu parametri, meno layer

I modelli Mixture of Experts (DeepSeek-V3, Mixtral) hanno meno layer di un denso comparabile, ma ogni layer contiene molti "esperti" FFN, di cui solo pochi sono attivati per ciascun token. Il risultato: piu parametri totali ma meno FLOPs per token, con un numero di layer contenuto (50-70).

Takeaway

I modelli densi di frontiera usano tra 80 e 130 Transformer Block. Il conteggio dei parametri scala con il prodotto tra profondità e quadrato della dimensione nascosta: oltre una certa soglia, si preferisce aumentare la larghezza per facilitare il parallelismo. I modelli MoE raggiungono un numero di parametri ancora maggiore con meno layer, distribuendo la capacità su molti esperti.

Inferenza Autoregressiva



$$\underbrace{\text{"Il", "gatto", "si"}}_{\text{3 token in input}} \rightarrow \text{LM} \rightarrow x_4?$$

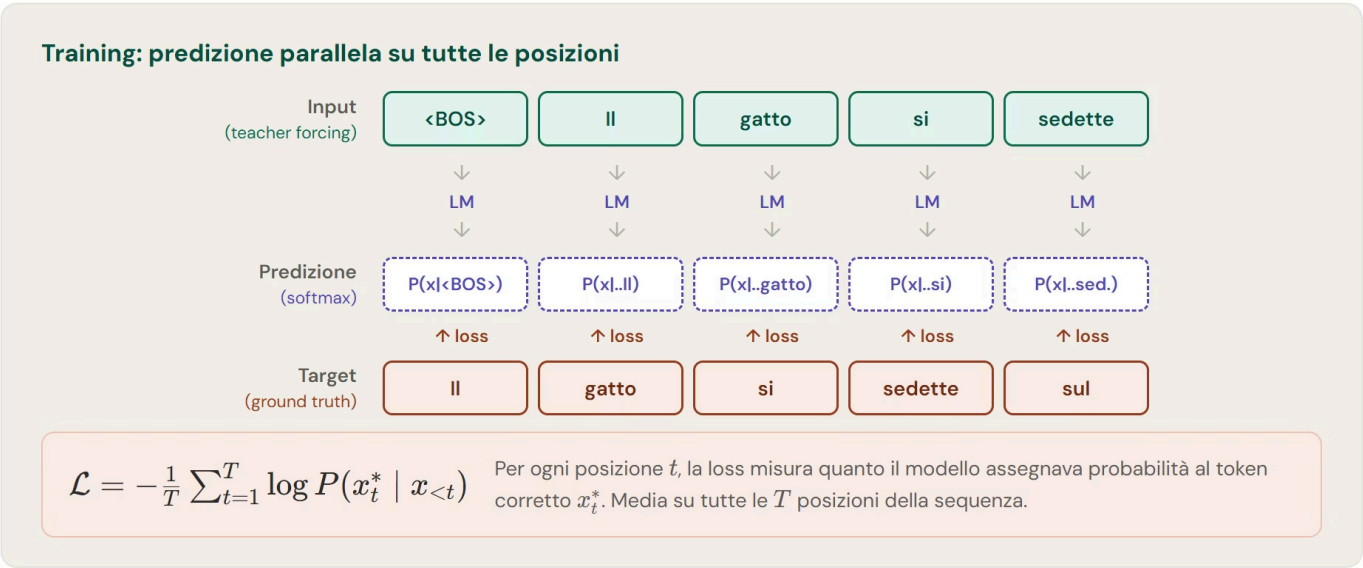
Step 1: il modello ha appena prodotto "si". Al prossimo step, riceverà tutti i **3 token** in input — non ricorda di averli già visti. L'intero contesto viene rielaborato da zero (nessun backtracking: la scelta è irreversibile).

Step 1: input = ["Il", "gatto"] → il modello predice "si". Il contesto è solo il prompt — la predizione è basata unicamente su queste 2 parole. Notare: il modello poteva scegliere "nero", "dorme", "corre"... la scelta è probabilistica.

[Inferenza](#)

Come impara il modello

Next-Token Prediction + Teacher Forcing + Cross-Entropy Loss



TRAINING VS INFERENZA

	Training	Inferenza
Input	Sequenza reale (teacher forcing)	Prompt + token generati dal modello stesso
Elaborazione	Tutte le posizioni in parallelo (causal mask)	Un token alla volta (autoregressivo)
Loss	Cross-entropy su ogni posizione	Nessuna -- solo sampling/argmax
Ground truth	Disponibile -- il target è il token successivo reale	Non disponibile -- il modello si basa sulle proprie scelte
Parallelismo	Alto -- un forward pass per l'intera sequenza	Basso -- N forward pass per N token generati

Teacher forcing = il modello in training riceve sempre l'input corretto, mai i propri errori. Questo accelera la convergenza, ma crea un **exposure bias**: in inferenza il modello deve gestire i propri output, una situazione mai vista in training. Il post-training (RLHF/RLVR) mitiga questo gap.

Strategie di Decoding

La LM Head produce una distribuzione di probabilità sul prossimo token. Il decoding decide *come scegliere* da quella distribuzione — e la scelta cambia radicalmente il comportamento del modello.

Il concetto chiave:

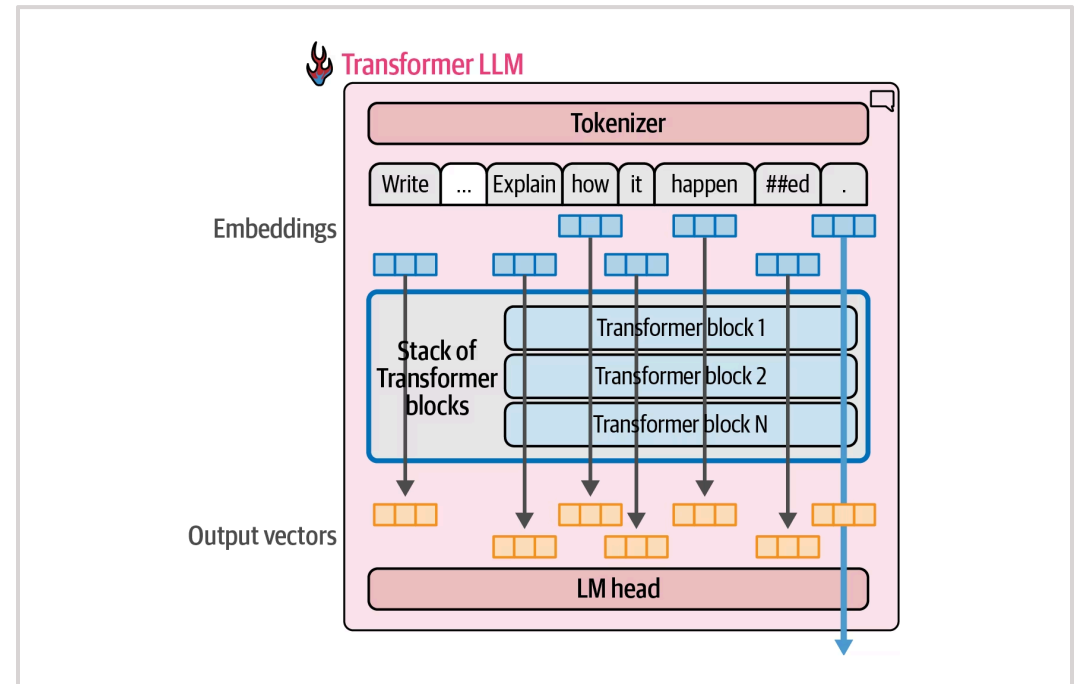
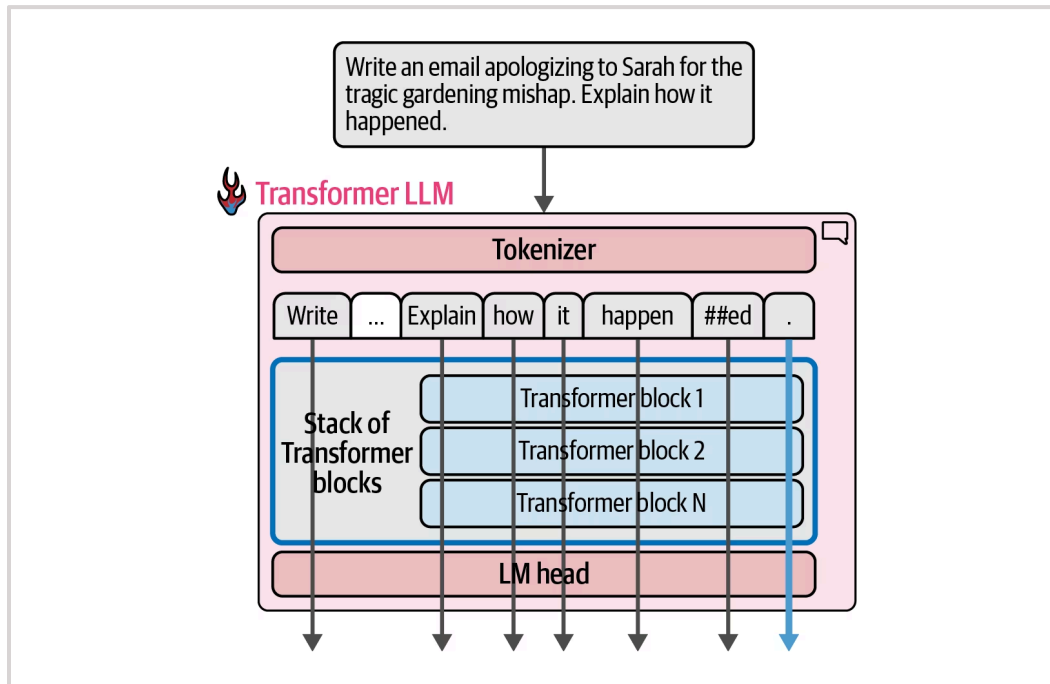
- La stessa distribuzione di probabilità può produrre output molto diversi a seconda della strategia
- Temperature, Top-K e Top-P non aggiungono conoscenza — redistribuiscono la probabilità già calcolata
- La strategia giusta dipende dal task: determinismo per il codice, variabilità per la scrittura creativa

Quale strategia usare?

- Codice, fatti, classificazione → T bassa / Greedy
- Chat, assistenza → T=0.7 + Top-P 0.9
- Scrittura creativa → T alta + Top-P 0.95
- Traduzione → Beam Search

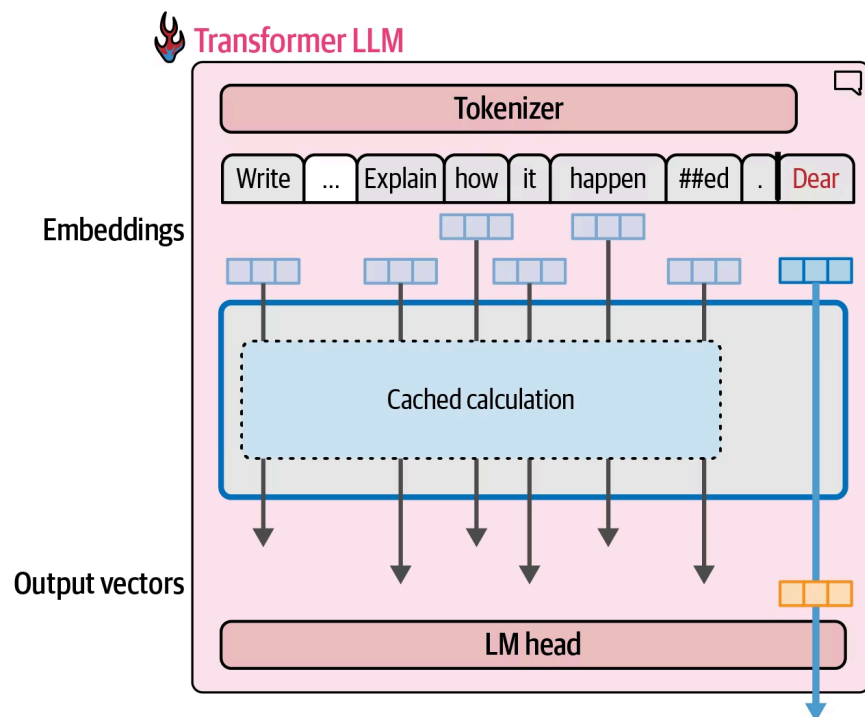
[Strategie di decoding](#)

Parallel Token Processing and Context Size



- **Elaborazione per posizioni in parallelo:** Durante la fase di *prefill*, i token della sequenza vengono elaborati in parallelo nelle diverse posizioni del contesto. La *context length* indica il numero massimo di posizioni/token gestibili dal modello, non il numero di pipeline fisiche distinte.
- **Dimensione uniforme delle rappresentazioni:** Ogni posizione è associata a un vettore di stato della stessa dimensione fondamentale (*hidden size* o *model dimension*) lungo i blocchi Transformer. I sottoblocchi interni possono usare dimensioni intermedie diverse, ma l'output del blocco torna alla dimensione del modello.
- **Ruolo dell'ultima posizione in generazione:** Nell'inferenza autoregressiva, il token successivo viene predetto a partire dai *logits* prodotti dall'ultima posizione della sequenza corrente tramite la LM head. In training, invece, i *logits* vengono prodotti per tutte le posizioni in parallelo.
- **Dipendenza dai token precedenti:** Anche se la previsione del prossimo token usa l'output dell'ultima posizione, tale rappresentazione dipende dai token precedenti tramite la *causal self-attention*. Per questo i token passati restano essenziali; in generazione incrementale, i loro contributi vengono tipicamente riutilizzati tramite *KV cache*.

KV Cache



- **Evitare calcoli ridondanti:** Quando si genera il secondo token, non è necessario ripetere i calcoli per il primo flusso, poiché i risultati precedenti possono essere salvati in memoria.
- **Caching dei vettori:** Il modello salva i vettori *Key* e *Value* (componenti del meccanismo di attenzione) calcolati nei passaggi precedenti.
- **Efficienza:** Grazie alla cache, per generare il token successivo è attivo solo l'ultimo flusso di elaborazione, riducendo drasticamente il tempo di calcolo.

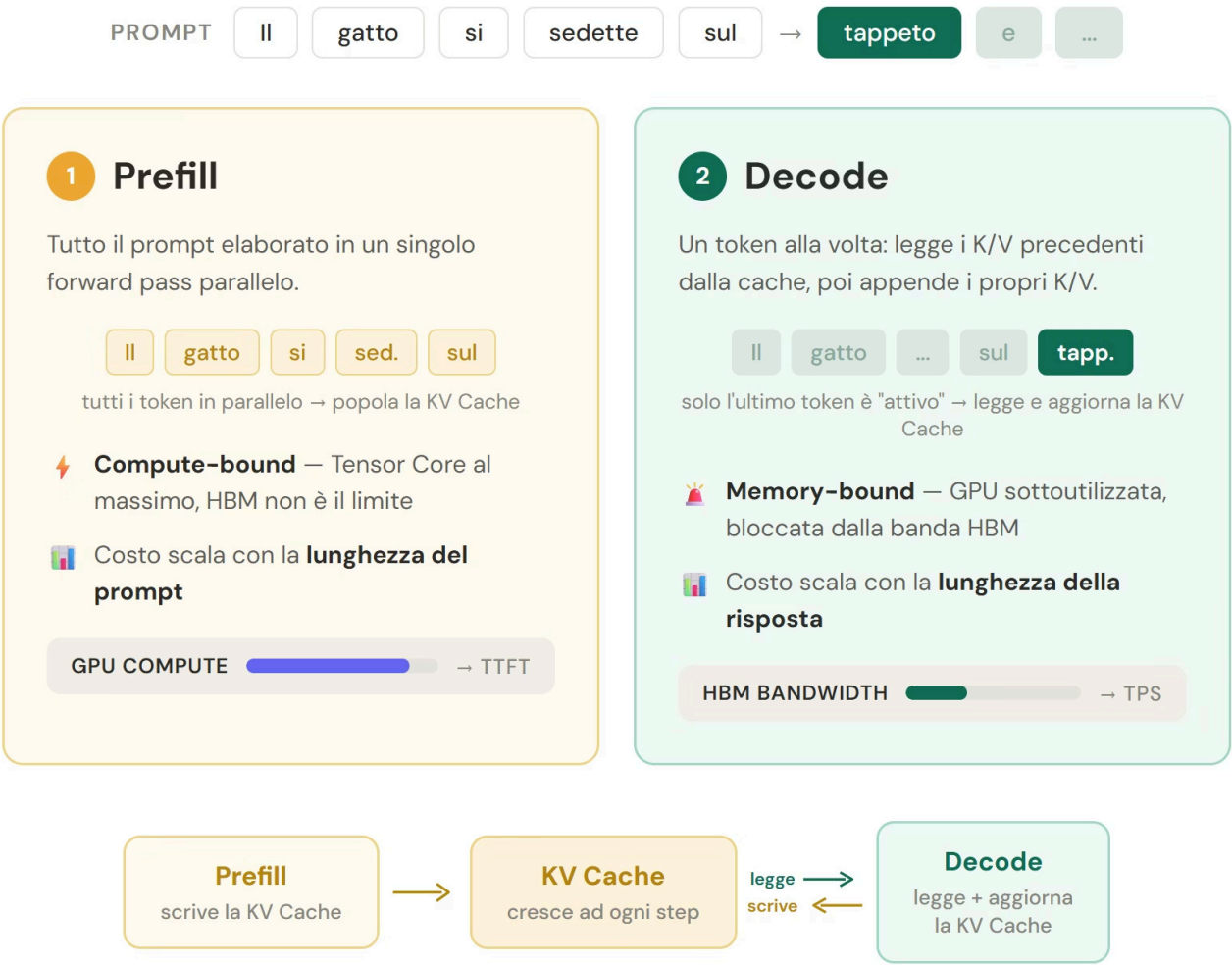
Questa tecnica è fondamentale per l'esperienza utente, permettendo ai sistemi LLM di generare testo in modo fluido e veloce.

Le due fasi dell'inferenza: Prefill e Decode

L'inferenza autogressiva non è un processo uniforme — si divide in due fasi con colli di bottiglia opposti.

Il punto chiave:

- Il prompt viene elaborato tutto insieme (prefill), la risposta viene generata un token alla volta (decode)
- Le due fasi hanno vincoli diversi: calcolo per il prefill, banda di memoria per il decode
- Questo spiega perché TTFT (Time To First Token) e TPS (Tokens Per Second) sono metriche indipendenti — e perché i provider hanno prezzi diversi per input e output token



Riepilogo: cosa cambia tra le due fasi

	PREFILL	DECODE
Token elaborati	Tutti in parallelo	Uno alla volta
Collo di bottiglia	Compute (FLOP)	Memoria (bandwidth)
Utilizzo GPU	Alto (Tensor Core saturi)	Basso (GPU sottoutilizzata)
KV Cache	Viene scritta	Viene letta
Metrica	TTFT (Time To First Token)	TPS (Tokens Per Second)
Dipende da	Lunghezza del prompt	Lunghezza della risposta

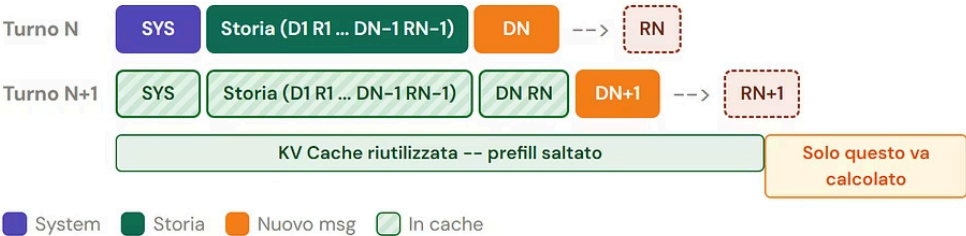
Prompt Caching

Riutilizzare la KV Cache tra richieste consecutive per evitare di ricalcolare il prefisso comune

Il problema

Il modello è stateless: ad ogni turno l'intera conversazione viene re-inviata. Il **prefill** ricalcola i vettori K/V per tutti i token di input, anche quelli identici al turno precedente. Al turno 10, oltre il 90% del prefill è lavoro già fatto.

Turno N vs Turno N+1: cosa cambia davvero?



Come funziona

- 1 Hash del prefisso:** il provider confronta i token di input con richieste recenti. Se il prefisso corrisponde, i K/V sono già in GPU.
- 2 Prefill parziale:** solo i token nuovi (ultimo messaggio utente) vengono elaborati. Il resto è letto dalla cache.
- 3 Decode invariato:** la generazione procede normalmente. Risultato matematicamente identico.

Impatto su costo e latenza

SCENARIO	PREFISSO IN CACHE	RISPARMIO
Chat turno 1	● Solo system	Basso
Chat turno 10+	➤ >90% token	-90% TTFT
Batch stesso SYS	➤ Prefisso condiviso	-50/75% costo

SUPPORTATO DA: Anthropic OpenAI Google DeepSeek

La KV Cache accelera la generazione *dentro* una richiesta. Il prompt caching riutilizza quella cache *tra* richieste consecutive -- trasformando la natura stateless del modello da problema economico a costo gestibile.

MHA vs MQA vs GQA – Ottimizzare la KV Cache

La KV Cache è il collo di bottiglia dell'inferenza — non per il calcolo, ma per la memoria. Tre architetture bilanciano espressività ed efficienza condividendo K e V tra le teste di attenzione.

Il problema:

- In MHA ogni testa ha la propria coppia K, V $\rightarrow$ la cache scala con h (numero di teste)
- Per un modello 70B con contesto 4096, la cache occupa ~4 GB per singola richiesta
- Con decine di richieste in parallelo, la memoria GPU si satura prima della potenza di calcolo

Le tre strategie:

- **MHA** (Multi-Head): h coppie K/V indipendenti — massima espressività, massima cache
- **MQA** (Multi-Query): 1 sola coppia K/V condivisa tra tutte le teste — cache minima, possibile degrado
- **GQA** (Grouped-Query): g gruppi di teste condividono ciascuno una coppia K/V — lo sweet spot

Il trend dei modelli moderni: GQA con 4-8 head per gruppo KV (Llama 2/3, Mistral, Gemma 2)

[Confronto](#)

Flash Attention

L'attenzione standard è limitata dalla memoria, non dal calcolo. FlashAttention riorganizza il calcolo per lavorare a blocchi nella SRAM veloce della GPU — risultato esatto, 2-4× più veloce.

Il problema:

- La matrice di attenzione $N \times N$ per 4096 token occupa 64 MB — non entra nella SRAM on-chip (~20 MB)
- L'attenzione standard la scrive e rilegge dalla HBM (lenta) più volte → il tempo è dominato dai trasferimenti dati, non dal calcolo

La soluzione:

- Divide Q, K, V in blocchi (tile) che entrano nella SRAM
- Calcola l'attenzione localmente blocco per blocco, senza mai materializzare la matrice $N \times N$
- L'online softmax permette di accumulare il risultato incrementalmente — nessuna approssimazione

Oggi è lo standard de facto in PyTorch, HuggingFace, vLLM e TensorRT-LLM

[Infografica](#)

Paged Attention

La KV Cache tradizionale pre-alloca blocchi contigui per la lunghezza massima — sprecando il 60-80% della memoria GPU. PagedAttention applica il principio della memoria virtuale: blocchi non contigui, allocazione dinamica, zero spreco.

L'analogia con il sistema operativo:

- Richiesta ↔ processo, blocco KV ↔ pagina virtuale, GPU memory ↔ RAM fisica, block table ↔ page table
- Ogni richiesta vede una cache logicamente contigua, ma fisicamente i blocchi sono sparsi nella GPU

I risultati:

- Frammentazione <4% (da ~60-80% nell'allocazione tradizionale)
- 2-4x più richieste servibili in parallelo con la stessa GPU
- Abilita il continuous batching e la condivisione del prefisso (copy-on-write per system prompt comuni)
- È il cuore di [vLLM](#), lo standard de facto per il serving di LLM in produzione

[Infografica](#)

Batching & Padding

Le GPU richiedono tensori rettangolari, ma le richieste degli utenti hanno lunghezze diverse — il padding risolve il problema, l'attention mask evita che inquina il risultato.

Il meccanismo:

- Richieste multiple vengono assemblate in un unico tensore (batch) per massimizzare l'utilizzo della GPU
- Le sequenze più corte vengono riempite con token vuoti (padding) fino alla lunghezza massima del batch
- L'attention mask "spegne" i token di padding così che il modello elabori solo i dati reali

Il dettaglio chiave:

- Nei principali serving stack autoregressivi (vLLM, TGI, HuggingFace generate) si usa tipicamente **left-padding**: il riempimento va a sinistra, così l'ultimo token reale di ogni sequenza è allineato alla fine — esattamente dove il modello predice il next token. Nota: non è un vincolo architetturale ma una convenzione pratica che semplifica l'estrazione dei logits dall'ultima posizione in batch.
- In training si usa right-padding (il contrario) perché il modello deve predire ogni posizione, non solo l'ultima

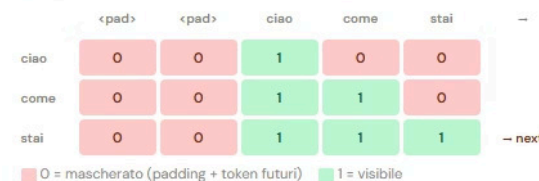
1 RICHIESTE IN INGRESSO (LUNGHEZZE DIVERSE)



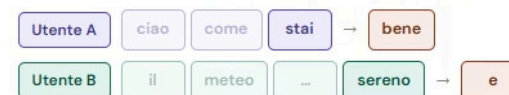
2 TENSORE CON LEFT-PADDING (2 × 5)



3 ATTENTION MASK (RIGA 0 — UTENTE A)



4 PREDIZIONE NEXT TOKEN (DALL'ULTIMA COLONNA)



— Left-padding (inferenza) —

Padding a sinistra. L'ultimo token reale è sempre nell'ultima colonna → facile estrarre la predizione dalla LM Head.

Right-padding (training) —

Padding a destra. La loss va calcolata su ogni posizione; la loss mask esclude il padding alla fine.

Continuous Batching

Il batching statico spreca GPU perché tutte le richieste devono finire insieme — il continuous batching risolve schedulando a livello di singola iterazione.

Il problema:

- Nel batching statico, se una richiesta genera 10 token e un'altra 500, la GPU resta inattiva per la prima fino a che la seconda non termina
- Più il batch è eterogeneo, più calcolo si butta via in attesa

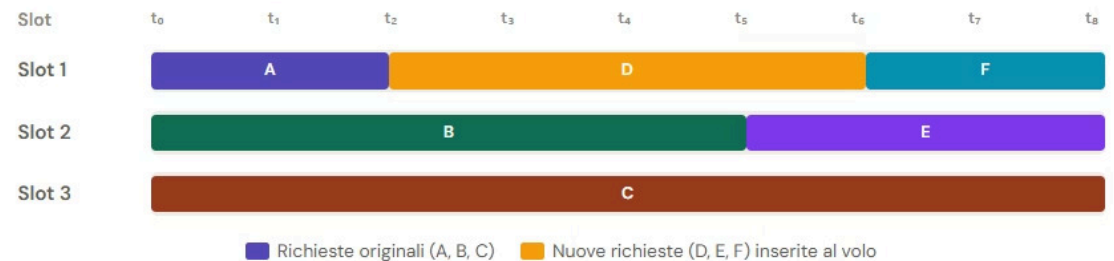
La soluzione:

- Appena una richiesta genera `<EOS>`, viene rimossa e una nuova entra al suo posto — nello stesso step di decode
- La GPU non è mai inattiva in attesa di richieste "lente"
- Insieme a Paged Attention, è il cuore di vLLM e TGI — i due serving engine più usati in produzione

● BATCHING STATICO



● CONTINUOUS BATCHING



● Batching Statico

3 richieste servite in 8 step.
~40% GPU idle su slot A e B.

● Continuous Batching

6 richieste servite nello stesso tempo.
Zero slot sprecate.

Paged Attention + Continuous Batching si abilitano a vicenda: le pagine dinamiche rendono economico inserire/rimuovere richieste dal batch ad ogni step. Insieme sono il cuore di vLLM e TGI.

Speculative Decoding

Un modello piccolo propone, il modello grande verifica — più token per forward pass, qualità identica.

L'intuizione:

- Il decode genera un solo token per forward pass, con la GPU sottoutilizzata (memory-bound)
- Per molti token la scelta è "facile" — un modello piccolo e veloce (draft) indovinerebbe la stessa risposta del modello grande (target)
- Il draft propone k token candidati, il target li verifica tutti in un singolo forward pass parallelo ($\approx$ un prefill)
- I token accettati vengono tenuti; dal primo rifiutato si ricampiona dal target $\rightarrow$ output **matematicamente identico**

Il risultato:

- Speedup di 2-3 $\times$ senza alcun degrado di qualità
- Task prevedibili (codice boilerplate, testo formulaico) $\rightarrow$ alto acceptance rate $\rightarrow$ grande speedup
- Task creativi o di ragionamento $\rightarrow$ acceptance rate più basso $\rightarrow$ speedup minore



Output matematicamente identico al target model da solo

Non è un'approssimazione — il rejection sampling garantisce la correttezza

Acceptance rate per tipo di task



Varianti recenti

Self-Speculative

Usa i primi layer del modello stesso come draft — nessun modello aggiuntivo necessario.

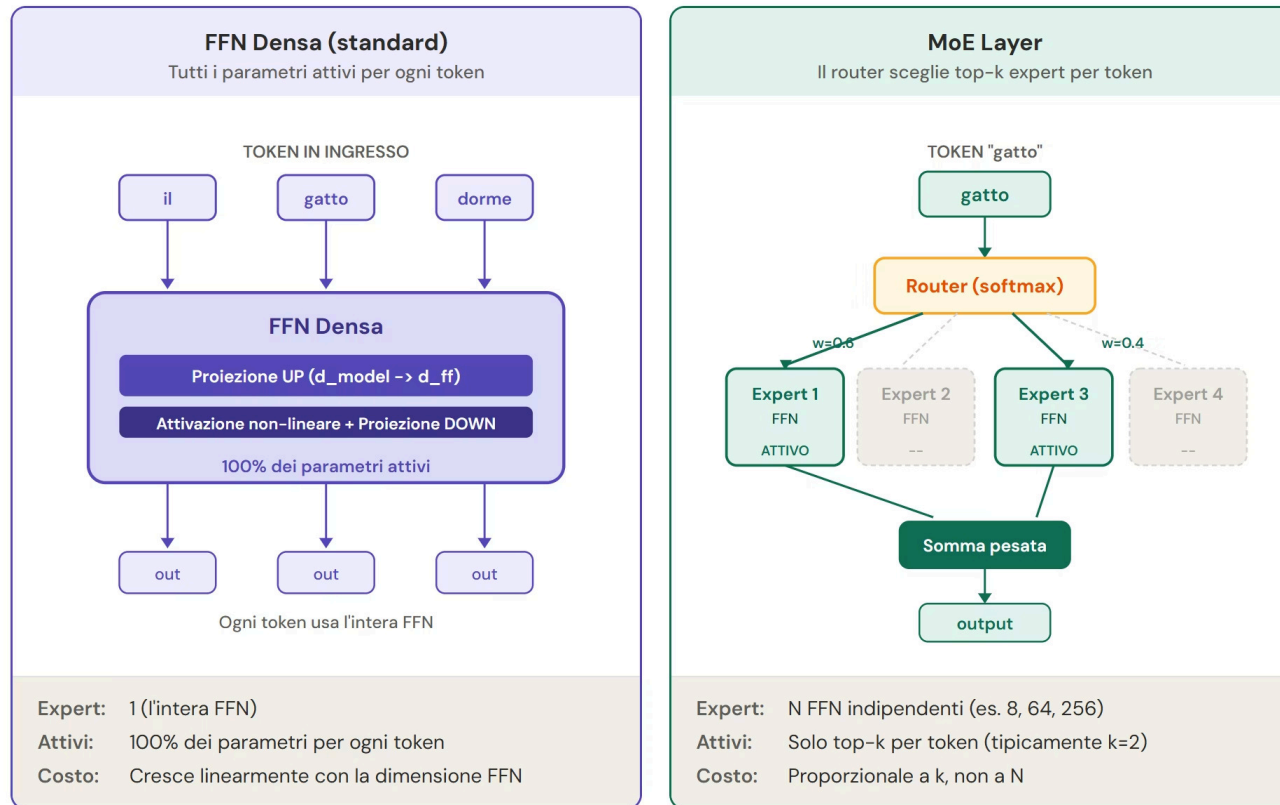
Medusa

Teste di predizione parallele aggiunte al target — prevede token a distanze diverse ($t+1$, $t+2$...).

Eagle

Predice nello spazio degli embedding anziché dei token — acceptance rate più alto.

Mixture of Experts (MoE)



MODELLI MOE IN PRODUZIONE

DeepSeek-V3

671B parametri totali, ~37B attivi per token
256 expert, top-8 attivati per token
I primi layer usano FFN dense (non MoE)

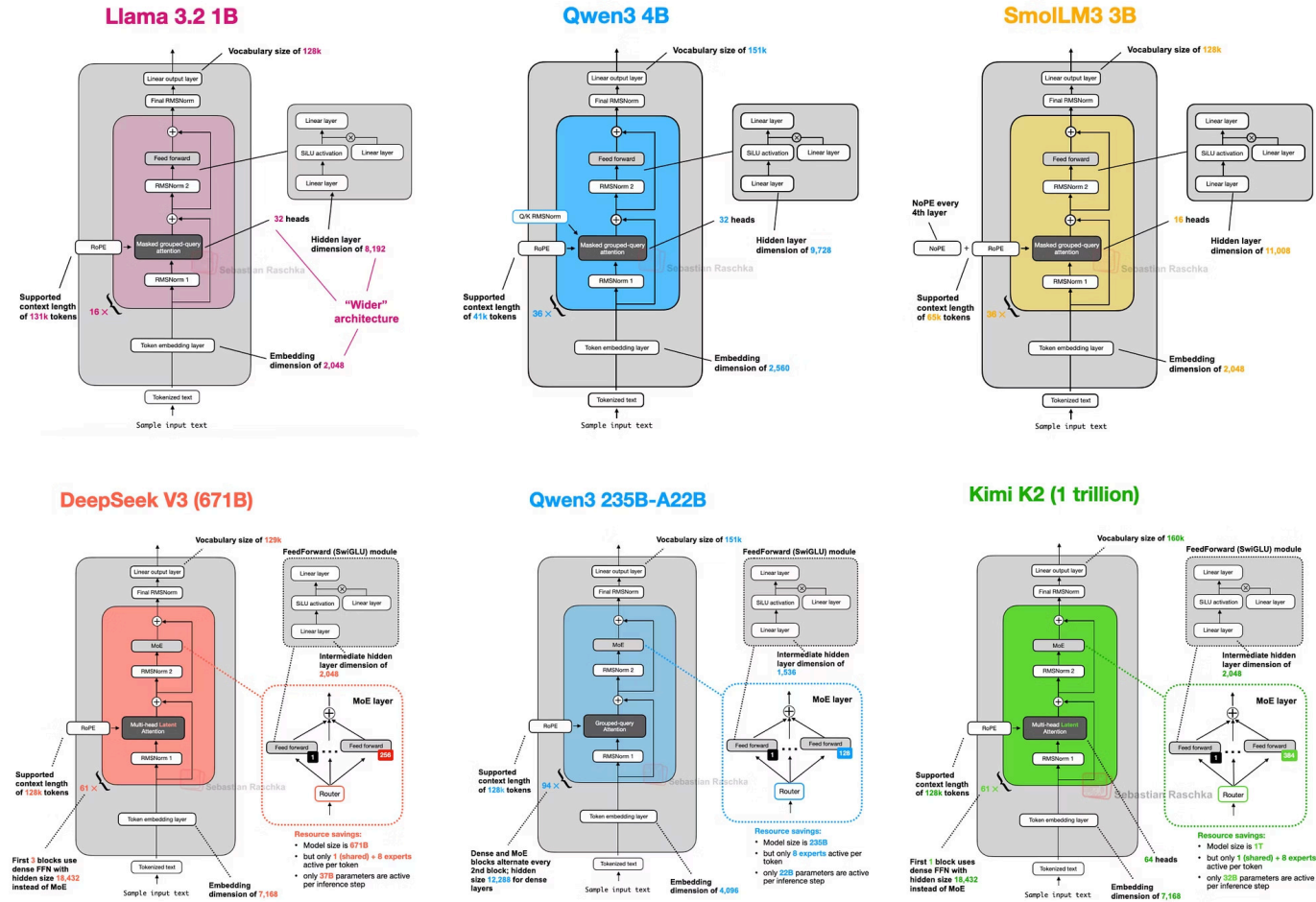
Mixtral 8x7B

46.7B parametri totali, ~12.9B attivi per token
8 expert per layer, top-2 attivati per token
Attention condivisa, solo FFN con expert

Ogni expert è una **FFN completa e indipendente** con i propri pesi -- non si tratta di "spegnere neuroni" in una singola FFN. Il router decide **quale FFN eseguire**, non quali neuroni attivare. La self-attention resta densa e condivisa: solo la FFN diventa sparsa.

[A Visual Guide to Mixture of Experts \(MoE\).](#)

The Big LLM Architecture Comparison



The Big LLM Architecture Comparison

Hands On

Riferimenti

Paper fondamentali:

- Bahdanau, D., Cho, K., & Bengio, Y. — *Neural Machine Translation by Jointly Learning to Align and Translate* (2014) — il paper che introduce il meccanismo di attention → arxiv.org/abs/1409.0473
- Vaswani, A. et al. — *Attention Is All You Need* (2017) — il paper originale del Transformer → arxiv.org/abs/1706.03762

Libri:

- Raschka, S. — *Build a Large Language Model (From Scratch)*, Manning, 2024 — guida pratica alla costruzione di un LLM passo per passo → manning.com | [GitHub](https://github.com)
- Amidi, A. & Amidi, S. — *Super Study Guide: Transformers & Large Language Models* — compendio visuale con ~600 illustrazioni, basato sul workshop Stanford CME 295 → superstudy.guide | [GitHub](https://github.com)

Risorse visive e interattive:

- Bycroft, B. — *LLM Visualization* — visualizzazione 3D interattiva di ogni layer di un LLM (nanoGPT) → bbycroft.net/llm
- Cho, A. et al. (Georgia Tech, Polo Club) — *Transformer Explainer* — spiegazione visiva e interattiva del Transformer con GPT-2 live nel browser → poloclub.github.io/transformer-explainer
- Raschka, S. — *A Visual Guide to Attention Variants in Modern LLMs* — panoramica visuale delle varianti di attention (MHA, GQA, MLA, ibridi) → magazine.sebastianraschka.com

Approfondimenti tecnici:

- Razvant, A. — *Understanding LLM Inference* (The AI Merge, 2025) — analisi del processo di inferenza, pre-fill e generation, configurazioni dei modelli su HuggingFace → read.theaimerge.com